ADBMS Practice Question for Practical Exam

1. A library wants to manage books and borrowers. Each book has Book_ID, Title, Author, and Price. Each borrower has Member_ID, Name, and Department. A member can borrow several books.

1. Identify the entities, attributes, and relationships, and draw an ER diagram.
2. Create Book and Member tables using SQL.
3. Insert two records into each table and display them.
4. Display books by a specified author.
5. Display books priced below 800.
6. Count the books written by each author.
7. Create a Borrowing table to record Member_ID, Book_ID, Issue_Date, and Return_Date.
8. Display the titles of books currently borrowed by a specified member.
9. Find members who have borrowed more than one book.
10. Update the price of a specified book and display the updated record.

2. A medical group has branches in Thane and Nashik. Each branch stores patient records for local access. Patient details include Patient_ID, Patient_Name, Age, and Branch.

1. Create a Patient table and insert four records for the two branches.
2. Display patients belonging to the Thane branch.
3. Create a second table for replicated patient records and copy selected records using INSERT.
4. Display the number of patients at each branch.
5. Find records in the original table that are missing from the replicated table.
6. Create a Branch table containing Branch_ID and Branch_Name, if required for the design.
7. Update the branch of a specified patient in the original table.
8. Copy all Nashik branch patient records into the replicated table.
9. Use a suitable query to display patient IDs present in both tables.
10. Explain briefly how replication improves local data availability and mention one consistency concern.

3. A university stores student information in MongoDB. Each student document contains Student_ID, Name, Department, Semester, and City.

1. Create a database named CollegeDB and a collection named students.
2. Insert three student documents.
3. Display students belonging to the Computer Science department.
4. Update the semester of one student.
5. Display the updated document.
6. Display all students in descending order of Name.
7. Find students whose Semester is greater than 2.

8. Display the number of students in each Department using an aggregation pipeline.
9. Remove one specified student document using Student_ID.
10. Find students living in a specified city.

4. A clinic stores patient details as objects. Each patient has Patient_ID, Patient_Name, Age, and an Address containing Street and City.

1. Create an Address_Type object type.
2. Create a Patient_Type object type containing patient details and an address object.
3. Create an object table for patients.
4. Insert two patient records and display each patient's name and city.
5. Display Patient_ID and Age for all patients.
6. Display Patient_Name, Street, and City using object attribute dot notation.
7. Insert a patient whose address is in a different city.
8. Display the complete object details of every patient.
9. Update the city attribute for one patient address.
10. Count the number of patient objects stored in the object table.

5. A university records students and their programmes using Datalog. Each student is associated with a department.

1. Define facts for four students.
2. Define facts associating students with departments.
3. Execute a query to display all students.
4. Execute a query to display students in the Data Science department.
5. Define and execute a rule to retrieve students in the Information Technology department.
6. Display students belonging to either Data Science or Information Technology.
7. Define a fact associating each student with a student ID.
8. Write a query to retrieve students whose department is not Data Science, using the supported Datalog syntax.
9. Create a rule to identify students belonging to the same department.
10. Query all students who belong to a department for which a fact has been defined.

6. A retail business stores transaction information with Sale_ID, Product_Name, Quantity, and Amount. The manager wants to compare product performance.

1. Create a Sales table and insert four records.
2. Calculate the total sales amount.
3. Calculate the total quantity sold for each product.
4. Display the product with the lowest total quantity sold.
5. Display products whose total sales amount exceeds 7,500.
6. Display products in descending order of total sales amount.

7. Calculate the average sale amount for each product.
8. Display the product with the highest total sales amount.
9. Count the number of sales transactions for each product.
10. Find the sale records whose Amount is greater than the overall average amount.

7. An online shop stores products in XML. Each product has Product_ID, Name, Category, and Price.

1. Create an XML document containing four products.
2. Write an XQuery to display all product names.
3. Retrieve products in the Clothing category.
4. Display products costing less than 8,000.
5. Display product names and prices in descending order of price.
6. Count the number of products in the Electronics category.
7. Retrieve all products with a price between 1,000 and 5,000.
8. Display product names for a specified category.
9. Calculate the average price of all products using XQuery.
10. Return the product details for the most expensive product.

8. A university stores spatial coordinates for its administration building, cafeteria, and sports complex.

1. Create a Campus_Place table with Place_ID, Place_Name, and a spatial Location column.
2. Insert three point locations using SDO_GEOMETRY.
3. Display the X and Y coordinates of all places.
4. Calculate the distance between the administration building and cafeteria using SDO_GEOM.SDO_DISTANCE.
5. Use SDO_WITHIN_DISTANCE to find places within 300 metres of the administration building.
6. Calculate the distance between the cafeteria and sports complex.
7. Create and populate USER_SDO_GEOM_METADATA for the spatial Location column.
8. Create a spatial index on the Location column, subject to the Oracle environment configuration.
9. Find the nearest campus place to the sports complex using distance calculations.
10. Display the names of places located within 1,000 metres of the cafeteria.

9. A college wants to check the integrity of student attendance records using hashing and demonstrate public-key cryptography.

1. Create a Python dictionary containing a student name and attendance percentage.

2. Generate a SHA-256 hash of the record.
3. Change the attendance percentage and generate a second hash.
4. Compare the hashes and display whether the record changed.
5. Generate an RSA public-private key pair and demonstrate encryption and decryption of a short record.
6. Verify whether a record matches its saved SHA-256 hash.
7. Write a Python function that accepts a record and returns its SHA-256 hash.
8. Save the original hash and compare it with a hash generated after changing the student name.
9. Demonstrate signing a short message with a private key and verifying it with the public key.
10. State the difference between hashing and encryption with respect to recovering the original data.

10. A college stores student information in a cloud SQL database. Each record contains Student_ID, Name, Department, and Semester.

1. Create a Student table and insert four records.
2. Display students in the Computer Science department.
3. Display students in Semester 3.
4. Count students in each department.
5. Display all students in alphabetical order of Name.
6. State one benefit of accessing the database through an online SQL interface.
7. Create a table named Department and insert at least three department records.
8. Write a query to display students from a specified city, adding City to the table if needed.
9. Update the semester of one student and verify the change.
10. Explain one security precaution when connecting to a cloud-hosted database.

11. An organisation stores employee details with Employee_ID, Employee_Name, Department, and Salary. Salary information should not be visible to every user.

1. Create an Employee table and insert three records.
2. Create a view that displays Employee_ID, Employee_Name, and Department but excludes Salary.
3. Grant SELECT permission on the view to an instructor-provided test user, subject to available privileges.
4. Display employees earning more than 60,000.
5. Revoke SELECT permission on the view from the test user.
6. Count the employees in each department.
7. Create a role for a test user, subject to available privileges.
8. Grant the minimum required SELECT privilege on the view to the role.
9. Attempt to query the base table as the test user and note the expected access restriction.

10. Explain why a view that omits Salary supports data confidentiality.

12. A college manages students and courses. Each student has Student_ID, Name, and Department. Each course has Course_ID, Course_Name, and Credits. Students may register for multiple courses.

1. Identify entities, attributes, and relationships, and draw an ER diagram.
2. Create Student and Course tables using SQL.
3. Insert three records into each table and display them.
4. Display students belonging to the Data Science department.
5. Display courses with at least 4 credits.
6. Display all courses in ascending order of credits.
7. Create a Registration table to represent the many-to-many relationship between Student and Course.
8. Insert at least three student-course registration records.
9. Display each student with the courses for which they registered.
10. Count the number of students registered for each course.

13. A university has campuses in Andheri and Vashi. Student records are maintained locally and selected records are copied to a reporting table.

1. Create a Student table with Student_ID, Name, Department, and Campus.
2. Insert four student records from both campuses.
3. Create a reporting table and copy selected student records into it.
4. Count students in each department at each campus.
5. Identify records in the original table that are missing from the reporting table.
6. Display reporting-table records ordered by Campus and Student_ID.
7. Create a Campus table or describe the key used to identify each campus.
8. Copy only Information Technology students into the reporting table.
9. Update a student name in the original table and identify whether the reporting copy is stale.
10. Write a query to display the total number of records in the original and reporting tables.

14. An online course platform stores student details and an array of enrolled courses. Each course entry contains Course_Name and Marks.

1. Create a database named LearningDB and a collection named learners.
2. Insert three student documents containing course arrays.
3. Find students enrolled in the Python course.
4. Update one student's marks in the Python course.
5. Display the updated document.
6. Display the names of students whose Python marks are greater than 75.

7. Find learners enrolled in more than one course using an array query.
8. Display learners whose marks in any course are greater than 80.
9. Add a new course entry to one learner document.
10. Count the number of learner documents in the collection.

15. A college represents faculty information using objects. Each faculty member has Faculty_ID, Name, and Specialization.

1. Create a Faculty_Type object type.
2. Create an object table named Faculty.
3. Insert three faculty records.
4. Display faculty names and specializations.
5. Insert a new faculty record using the Faculty_Type object constructor.
6. Display the total number of faculty members.
7. Add a Department attribute to the faculty object type if required and recreate dependent objects as needed.
8. Query faculty members in the Database Management specialization.
9. Update a faculty object record using a valid Oracle object-table operation.
10. Display faculty records in ascending order of Faculty_ID.

16. A college stores facts about subjects and instructors. Each instructor teaches one or more subjects.

1. Define facts for four subjects.
2. Define facts for two instructors and their teaching assignments.
3. Create a rule to derive instructor-subject relationships.
4. Query and display all derived relationships.
5. Display instructors teaching the subject Database Systems.
6. Retrieve all subjects taught by instructor Prof. Mehta.
7. Define facts that represent the department associated with each subject.
8. Write a rule to find instructors who teach more than one subject, using the supported Datalog syntax.
9. Query all instructors who teach at least one subject.
10. Add a new instructor assignment fact and rerun the relevant query.

17. A company records sales by department. Each sale has Sale_ID, Department, Product_Name, and Amount.

1. Create a Sales table and insert four records.
2. Calculate total sales for each department.
3. Display the department with the lowest total sales.
4. Display product names and total sales amounts in descending order of sales.

5. Display departments with total sales greater than 15,000.
6. Display the total number of sales records for each department.
7. Calculate the average sale amount for each department.
8. Display the department with the highest total sales.
9. Find products whose combined sales amount exceeds 5,000.
10. Calculate the percentage contribution of each department to overall sales, if supported by the SQL environment.

18. A college stores its subject catalogue in XML. Each subject contains Subject_ID, Subject_Name, Department, and Credits.

1. Create an XML document containing four subjects.
2. Write an XQuery to display all subject names.
3. Retrieve subjects offered by the Computer Science department.
4. Sort subjects by credits in descending order.
5. Display subjects having fewer than 4 credits.
6. Count the subjects offered by the Data Science department.
7. Retrieve subjects with credits between 3 and 5.
8. Display the names of subjects offered by a specified department.
9. Calculate the average credit value of all subjects using XQuery.
10. Return the subject with the maximum number of credits.

19. A healthcare network stores the locations of three clinics and wants to find nearby clinics.

1. Create a Clinic table with Clinic_ID, Clinic_Name, and a spatial Location column.
2. Insert three clinic locations using SDO_GEOMETRY.
3. Display clinic names and coordinates.
4. Calculate the distance between Clinic A and Clinic B.
5. Use SDO_WITHIN_DISTANCE to find clinics within 5,000 metres of Clinic A.
6. Calculate the distance between Clinic B and Clinic C using SDO_GEOM.SDO_DISTANCE.
7. Insert a fourth clinic location and update the spatial metadata extent if required.
8. Create a spatial index on the clinic Location column, subject to the Oracle environment configuration.
9. Find the clinic nearest to Clinic C using distance calculations.
10. Use SDO_RELATE or another supported spatial predicate to demonstrate a spatial relationship between two geometries.

20. An examination office stores student marks and wants to verify that result records have not been altered using a Merkle tree.

1. Create four sample result records in Python.

2. Generate a SHA-256 hash for each result record.
3. Combine the hashes to calculate a Merkle root.
4. Modify one mark and recalculate the Merkle root.
5. Verify a result record against its stored hash.
6. Construct a Merkle tree from eight result records and display the root.
7. Combine the eight leaf hashes pairwise to show the intermediate hash levels of the Merkle tree.
8. Change a student name without changing marks and verify whether the corresponding record hash changes.
9. Compare the original and modified Merkle roots and state what the difference indicates.
10. Explain how a Merkle proof can be used to verify one result without comparing every record.

21. A library stores book details in a cloud database. Each book has Book_ID, Title, Author, and Price.

1. Create a Book table and insert four records.
2. Display books costing more than 700.
3. Display all books in ascending order of price.
4. Count the books written by each author.
5. Retrieve books written by a specified author.
6. Display the most expensive book.
7. Create a Category column or table and classify each book by category.
8. Display books whose price is between 500 and 1,000.
9. Update the price of a specified book and verify the update.
10. Explain one advantage and one risk of storing library data in a cloud database.

22. A company maintains employee records and wants to control access to employee information.

1. Create an Employee table with Employee_ID, Employee_Name, Department, and Salary, and insert three records.
2. Create a view that excludes the Salary column.
3. Grant SELECT permission on the view to an instructor-provided test user, subject to available privileges.
4. Display employees in the Sales department earning more than 45,000.
5. Revoke SELECT permission on the view from the test user.
6. Display the number of employees in each department.
7. Create a role for a test user, subject to available privileges.
8. Grant SELECT on the view to the role and describe how a user receives the role.
9. Write a query to display employees in a specified department without revealing salary.
10. Explain the principle of least privilege and how it applies to this employee database.